

	LEMBAR KERJA MANDIRI 5 COMPUTER VISION T.A. Genap 2025/2026	Nilai	Paraf Dosen

A. Identitas

Topik : Implementasi Object Detection dengan YOLO
 NIM : 2388010004
 Nama : Abdul Kharis
 Kelas : inf 6A

B. Tujuan

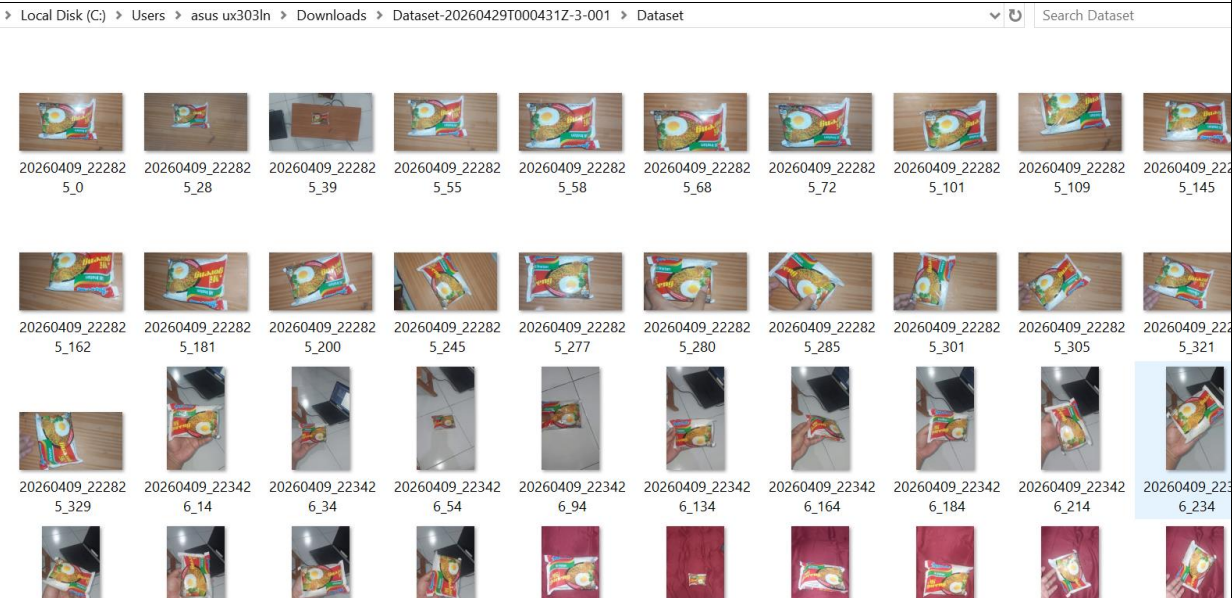
1. Mampu menjelaskan konsep object detection
2. Mampu melakukan instalasi anaconda
3. Mampu melakukan instalasi label studio
4. Melakukan labeling dataset gambar
5. Membuat mengimplementasikan YOLO sederhana

C. Instruksi Pengerjaan

• Tahap 1 – Install Anaconda

1. Download anaconda & Dataset Praktikum via website atau link drive berikut :
<https://drive.google.com/drive/folders/1wt7VCt9wKosr5e2SWtarOfNOvg3eVXJu?usp=sharing>
2. Install anaconda

Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan



instal **Anaconda** buat manajemen *library* biar nanti pas instal YOLO dan Label Studio nggak bentrok. Setelah instalasi selesai, saya lanjut nyiapin dataset foto Indomie yang sudah diunduh. Kalau dilihat di *screenshot* folder saya, semua file gambarnya sudah saya ekstrak ke direktori lokal supaya gampang dipanggil pas proses *labeling* nanti.

- **Tahap 2 – Install Label Studio**

1. Setelah install anaconda, buka **Anaconda Prompt**
2. Buat environment dengan nama **labelstudio310**

```
conda create -n labelstudio310 python=3.10 -y
```

Sintaks “Labelstudio310” adalah nama proyek yang dibuat oleh anaconda. Bisa diganti namanya dengan yang lain

3. Aktifkan

```
conda activate labelstudio310
```

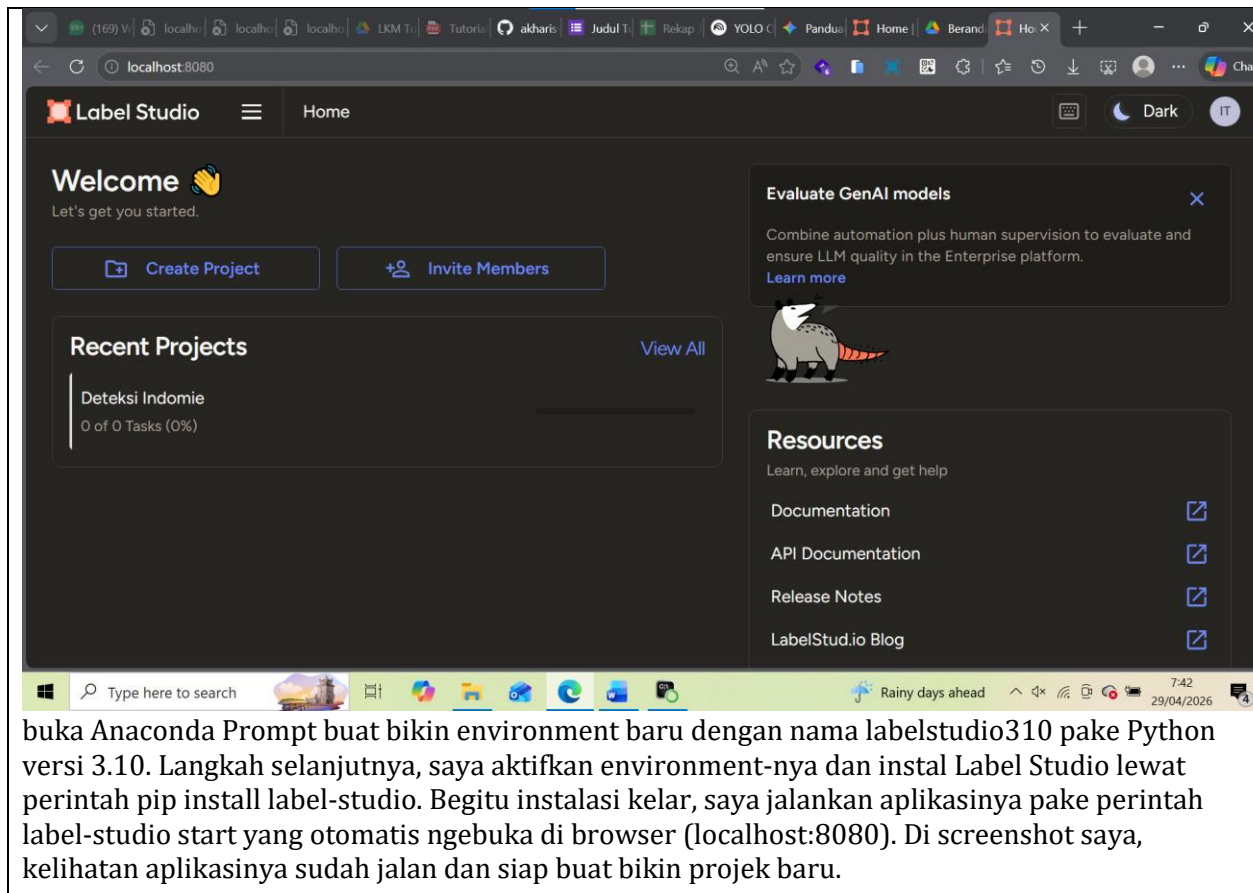
4. Install

```
pip install label-studio
```

5. Tunggu hingga selesai. Untuk menjalankannya yaitu dengan cara

```
label-studio start
```

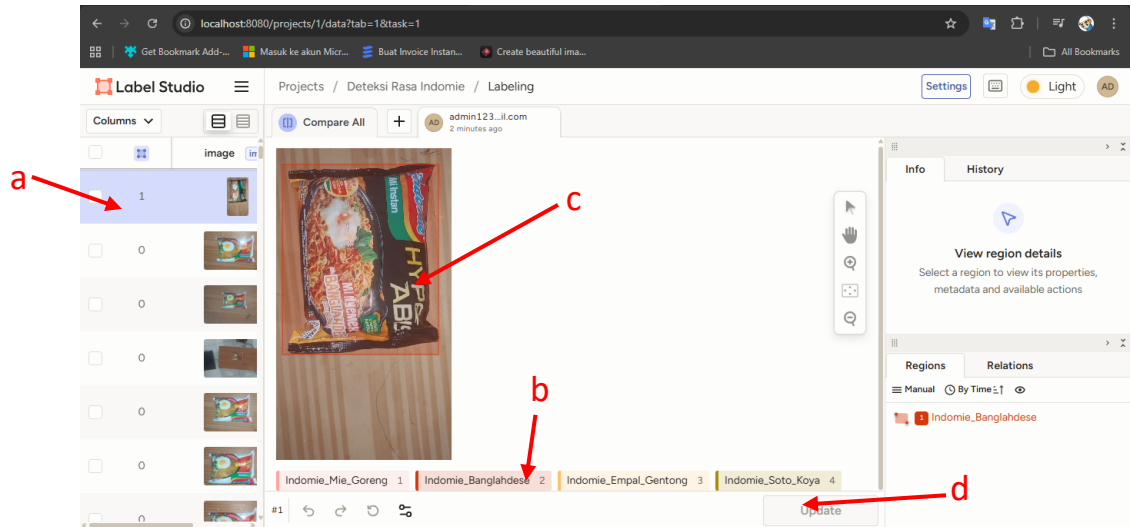
Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan



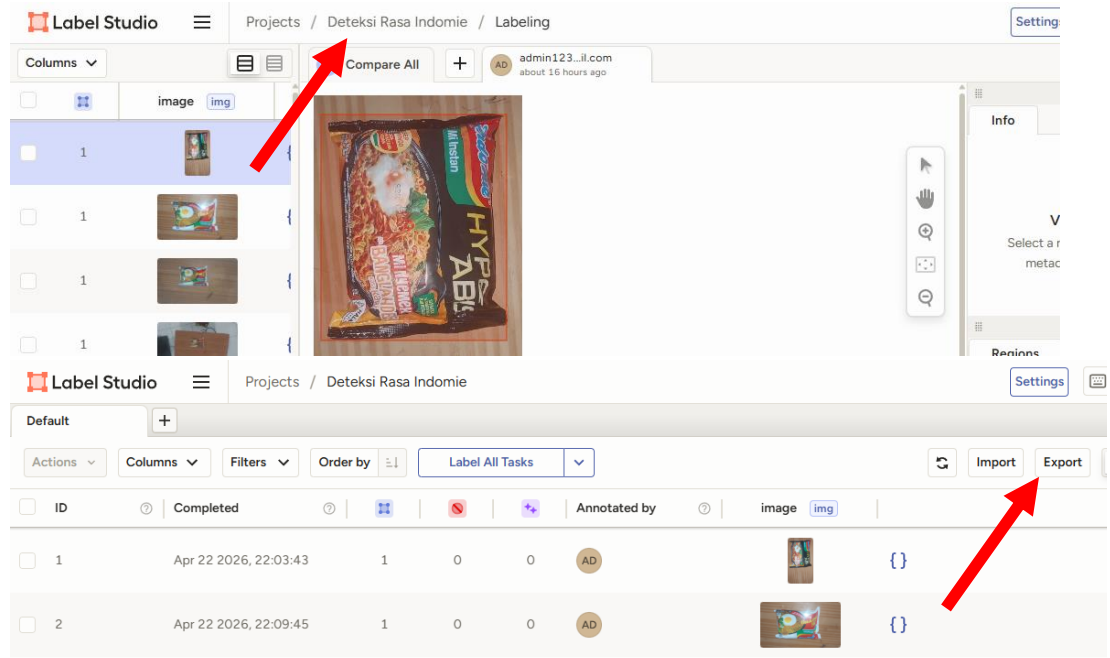
buka Anaconda Prompt buat bikin environment baru dengan nama labelstudio310 pake Python versi 3.10. Langkah selanjutnya, saya aktifkan environment-nya dan instal Label Studio lewat perintah pip install label-studio. Begitu instalasi kelar, saya jalankan aplikasinya pake perintah label-studio start yang otomatis ngebuka di browser (localhost:8080). Di screenshot saya, kelihatan aplikasinya sudah jalan dan siap buat bikin proyek baru.

• Tahap 3 – Labeling dataset

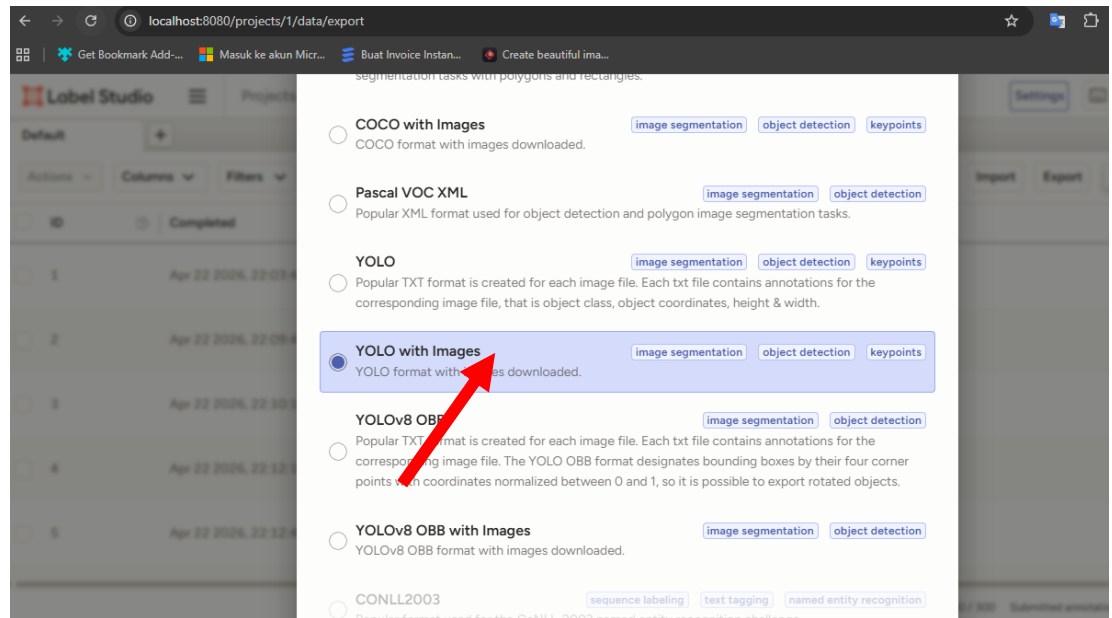
1. Setelah label Studio terbuka di browser, lakukan sign up
2. Log in
3. Create New Project
4. Dibagian Project Name -> beri nama object
5. Dibagian Data Import -> masukan dataset (upload secara bertahap, maksimal sekali upload adalah 100 gambar)
6. Dibagian Labeling Setup
 - a. Pilih Object Detection with Bounding Boxes
 - b. Hilangkan 2 label awal
 - c. Tambahkan label name
 - d. Save
7. Lakukan mengikuti urutan seperti gambar di bawah ini



- a. Pilih gambar yang mau dilabeli
 - b. Pilih jenis label
 - c. Buat Bounding Box
 - d. Klik Submit
 - e. Lakukan kesemua gambar dataset
8. Setelah semua gambar dataset diberikan label dengan bounding box, klik nama project -> export

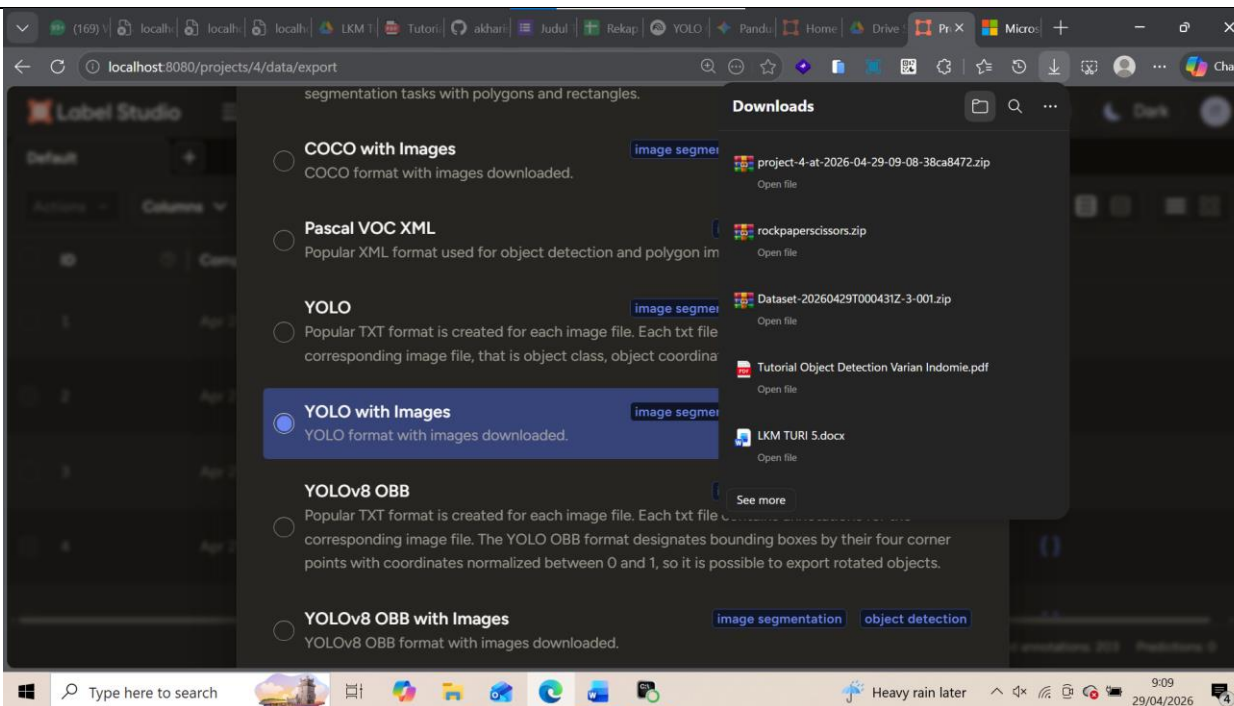


Lalu pilih
Yolo wigh Images



Pilih export

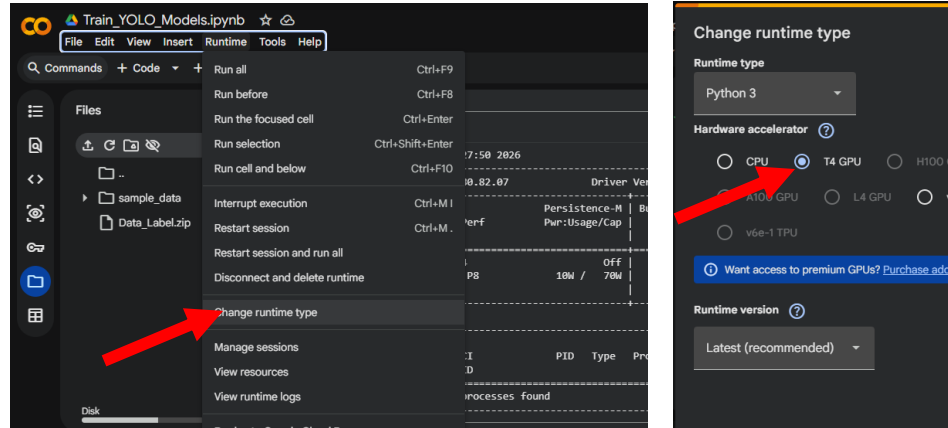
Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan



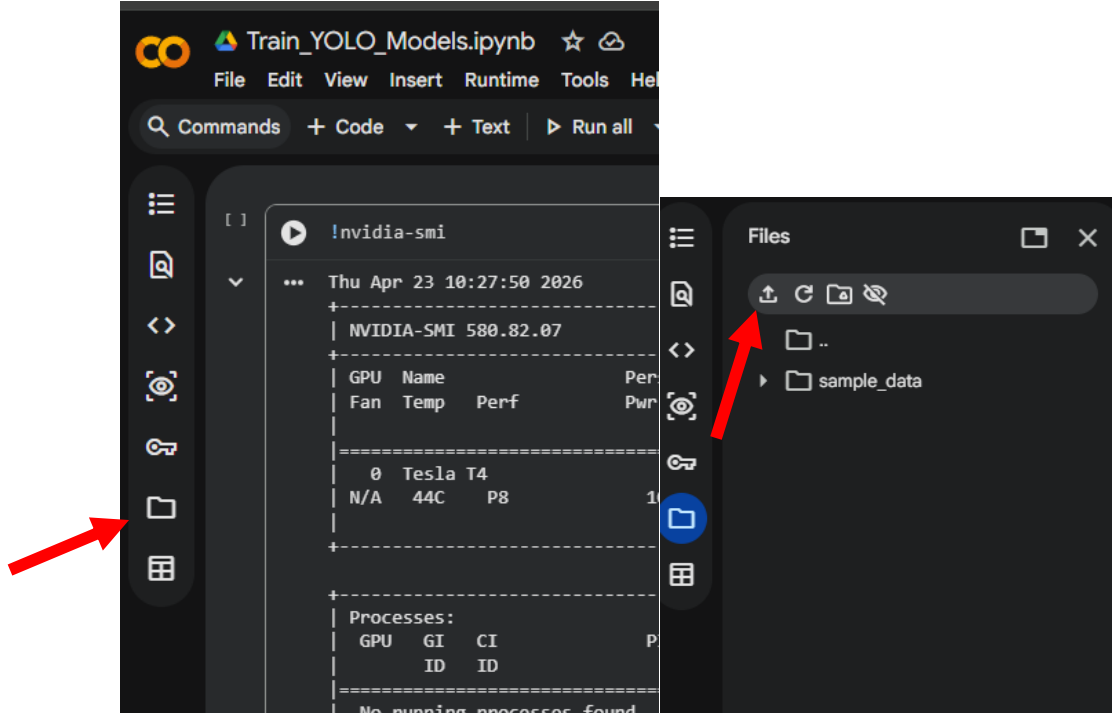
buka Label Studio di browser buat proses pelabelan foto Indomie yang sudah disiapkan. Pertama, bikin proyek baru dan *upload* semua gambarnya ke bagian Data Import. Di bagian Labeling Setup, saya pilih *Object Detection* dan buat label sesuai varian Indomie-nya, contohnya "Indomie_Banglahdese" dan "Indomie_Mie_Goreng". Setelah itu, mulai kasih *bounding box* satu-satu ke semua gambar buat nandain posisi objeknya. Kalau sudah selesai semua, hasilnya saya Export ke format YOLO dalam bentuk file zip supaya bisa dipakai buat proses *training* model nanti.

- **Tahap 4 – Implementasi YOLO**

1. Buat Project Google Collab terbaru (bisa diberi nama **Train_Yolo_Models**)
2. Ubah runtime klik menu Runtime -> Change runtime type menjadi T4 GPU



3. Upload file zip hasil dari label studio ke Google Collab (bisa di Rename terlebih dahulu file zipnya menjadi **Data_Label.zip**)



4. Unzip file yang dimasukan

```
1 !unzip -q /content/Data_Label.zip -d /content/custom_data
```

5. Gunakan sintaks dibawah ini untuk membagi dataset menjadi 90% untuk training dan 10% testing

```
!wget -O /content/train_val_split.py
https://raw.githubusercontent.com/EdgeElectronics/Train-and-Deploy-YOLO-Models/refs/heads/main/utils/train_val_split.py
```

```
!python train_val_split.py --datapath="/content/custom_data" --train_pct=0.9
```

6. Install library ultralytics

```
1 !pip install ultralytics
```

7. Membuat YAML file untuk konfigurasi training dengan Yolo nya. Bisa mengikuti sintaks dibawah ini:

```
1 import yaml
2 import os
3
4 def create_data_yaml(path_to_classes_txt, path_to_data_yaml):
5
6
7     if not os.path.exists(path_to_classes_txt):
8         print(f'classes.txt file not found! Please create a classes.txt labelmap
9         and move it to {path_to_classes_txt}')
10        return
11    with open(path_to_classes_txt, 'r') as f:
12        classes = []
13        for line in f.readlines():
14            if len(line.strip()) == 0: continue
15            classes.append(line.strip())
16        number_of_classes = len(classes)
17
18    data = {
19        'path': '/content/data',
20        'train': 'train/images',
21        'val': 'validation/images',
22        'nc': number_of_classes,
23        'names': classes
24    }
25
26
27    with open(path_to_data_yaml, 'w') as f:
28        yaml.dump(data, f, sort_keys=False)
29        print(f'Created config file at {path_to_data_yaml}')
30
31    return
32
33
34 path_to_classes_txt = '/content/custom_data/classes.txt'
35 path_to_data_yaml = '/content/data.yaml'
36
37 create_data_yaml(path_to_classes_txt, path_to_data_yaml)
38
39 print('\nFile contents:\n')
40 !cat /content/data.yaml
```


8. Training

```
1 !yolo detect train data=/content/data.yaml model=yolo11s.pt epochs=40 imgsz=640
```

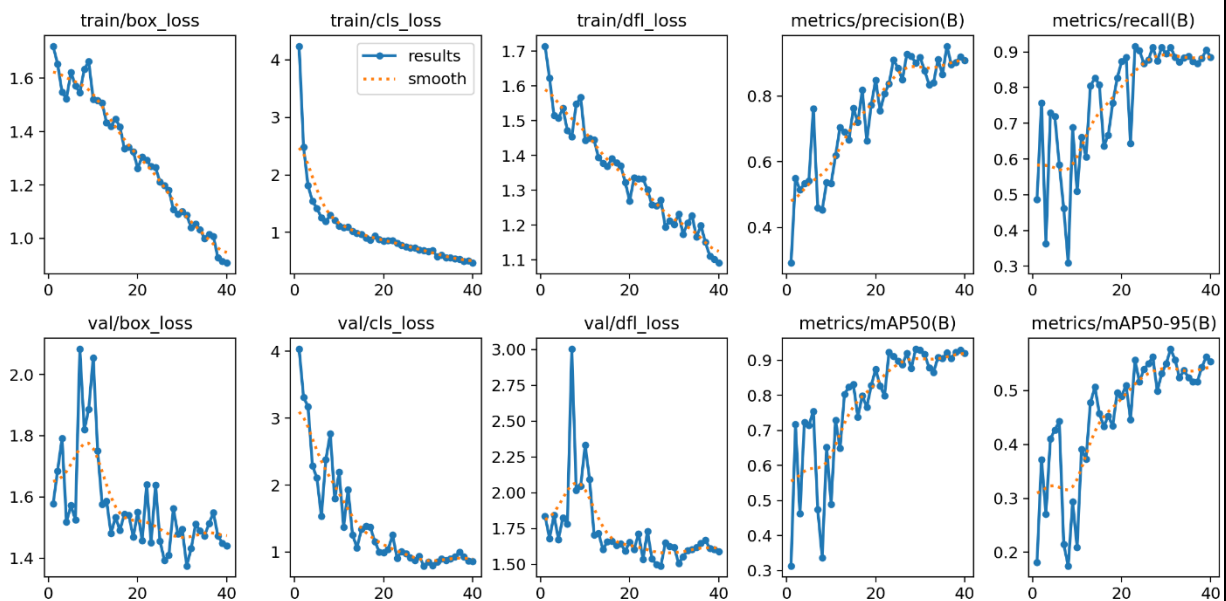
9. Testing

```
1 !yolo detect predict model=runs/detect/train/weights/best.pt
source=data/validation/images save=True
```

10. Prediksi dengan 10 gambar

```
1 import glob
2 from IPython.display import Image, display
3 for image_path in glob.glob(f'/content/runs/detect/predict/*.jpg')[:10]:
4     display(Image(filename=image_path, height=400))
5     print('\n')
6
```

Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan



proses *training* model menggunakan Google Colab dengan dukungan GPU T4 agar prosesnya lebih cepat.

- Persiapan: Saya mengunggah file Data_Label.zip hasil dari Label Studio, lalu mengekstraknya dan membagi dataset menjadi 90% untuk *training* serta 10% untuk validasi menggunakan skrip `train_val_split.py`.
- Konfigurasi & Training: Saya membuat file `data.yaml` untuk mengatur jalur folder gambar dan nama kelas objek, kemudian menjalankan proses *training* selama 40 epoch menggunakan model `yolo11s.pt`.
- Analisis Hasil: Berdasarkan grafik hasil *training* yang dilampirkan:

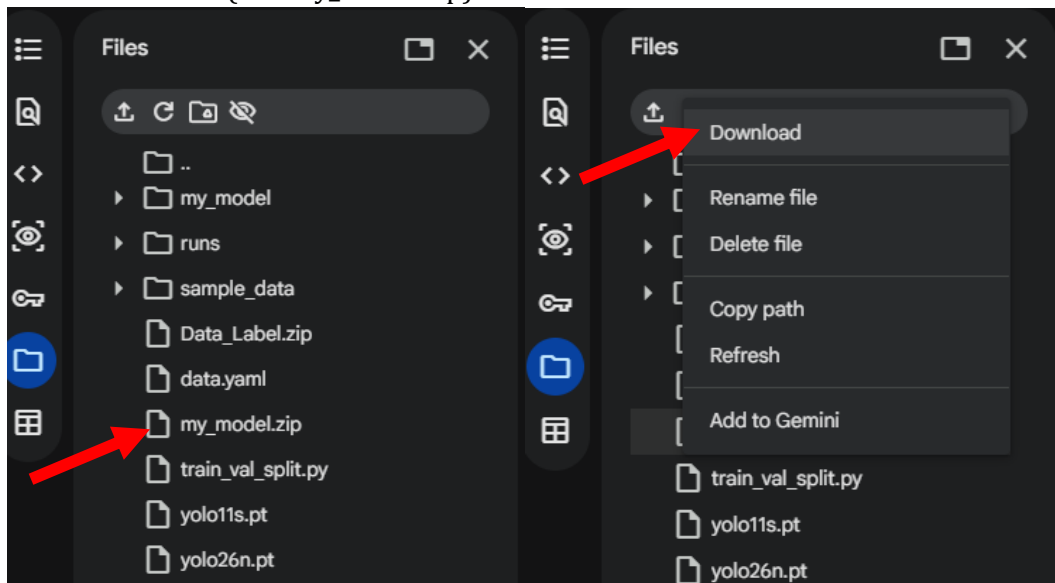
- Loss: Grafik *box_loss*, *cls_loss*, dan *dfl_loss* menunjukkan penurunan yang konsisten, artinya model semakin pintar dalam mengenali lokasi dan jenis objek.
- Metrik Akurasi: Grafik *Precision*, *Recall*, dan *mAP50* terus meningkat hingga mendekati angka 0.9 (90%), yang membuktikan bahwa model sudah sangat akurat dalam mendeteksi varian Indomie.

- **Tahap 5 – Deploy Model**

1. Buat folder "my_model" untuk menyimpan bobot model dan hasil pelatihan.

```
1 !mkdir /content/my_model
2 !cp /content/runs/detect/train/weights/best.pt /content/my_model/my_model.pt
3 !cp -r /content/runs/detect/train /content/my_model
4
5 %cd my_model
6 !zip /content/my_model.zip my_model.pt
7 !zip -r /content/my_model.zip train
8 %cd /content
```

2. Download Model (File my_model.zip)



Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan

```

[ ] ▶ !cp /content/runs/detect/train/weights/best.pt /content/my_model/my_model.pt
      !cp -r /content/runs/detect/train /content/my_model

      %cd my_model
      !zip /content/my_model.zip my_model.pt
      !zip -r /content/my_model.zip train
      %cd /content

  ▾ ... /content/my_model
      adding: my_model.pt (deflated 8%)
      adding: train/ (stored 0%)
      adding: train/BoxPR_curve.png (deflated 20%)
      adding: train/train_batch0.jpg (deflated 7%)
      adding: train/train_batch2.jpg (deflated 6%)
      adding: train/confusion_matrix.png (deflated 26%)
      adding: train/train_batch1.jpg (deflated 5%)
      adding: train/labels.jpg (deflated 32%)
      adding: train/val_batch0_pred.jpg (deflated 15%)
      adding: train/train_batch361.jpg (deflated 15%)
      adding: train/args.yaml (deflated 53%)
      adding: train/BoxF1_curve.png (deflated 9%)
      adding: train/train_batch362.jpg (deflated 16%)
      adding: train/results.png (deflated 6%)
      adding: train/train_batch360.jpg (deflated 15%)
      adding: train/BoxP_curve.png (deflated 12%)
      adding: train/confusion_matrix_normalized.png (deflated 24%)
      adding: train/BoxR_curve.png (deflated 12%)
      adding: train/results.csv (deflated 60%)
      adding: train/weights/ (stored 0%)
      adding: train/weights/best.pt (deflated 8%)
      adding: train/weights/last.pt (deflated 8%)
      adding: train/val_batch0_labels.jpg (deflated 16%)
      /content

```

Setelah *training* selesai, mennyiapkan hasil modelnya buat dipakai di perangkat lokal.

- Penyimpanan Hasil: bikin folder baru bernama `my_model` dan menyalin bobot model terbaik (`best.pt`) ke sana, sekalian saya ganti namanya jadi `my_model.pt` supaya lebih rapi.
- Pengarsipan: Semua file penting hasil pelatihan, termasuk grafik performa dan bobot model, dibungkus jadi satu file zip (`my_model.zip`) lewat perintah `!zip` di Colab.
- Download: Terakhir, file zip tersebut saya unduh ke laptop. File inilah yang nantinya diekstrak buat proses *deployment* deteksi kamera secara *real-time*.

• Tahap 6 – Deploy Model di Local Device

1. Unzip File `my_model.zip` dari hasil tahap sebelumnya
2. Dalam folder hasil unzip (folder `my_model`) buat file python dengan nama **`camera_detect.py`**. Sintaks sebagai berikut:

```

1  from ultralytics import YOLO
2  import cv2
3
4  model = YOLO("my_model.pt")
5
6  cap = cv2.VideoCapture(0)
7
8  while True:
9      ret, frame = cap.read()
10     if not ret:
11         break
12
13     results = model(frame, imgsz=640)
14
15     frame = results[0].plot()
16
17     cv2.imshow("Detection", frame)
18
19     if cv2.waitKey(1) & 0xFF == ord('q'):
20         break
21
22 cap.release()
23 cv2.destroyAllWindows()

```

3. Close **Anaconda Prompt** kemudian buka kembali
4. Aktifkan environment yang dibuat di Tahap 2

```
conda activate labelstudio310
```

5. Masuk ke Folder my_model berada. Contoh folder saya berada di directory D -> Folder my_model maka di **Anaconda Prompt** saya masukan

```
D:
```

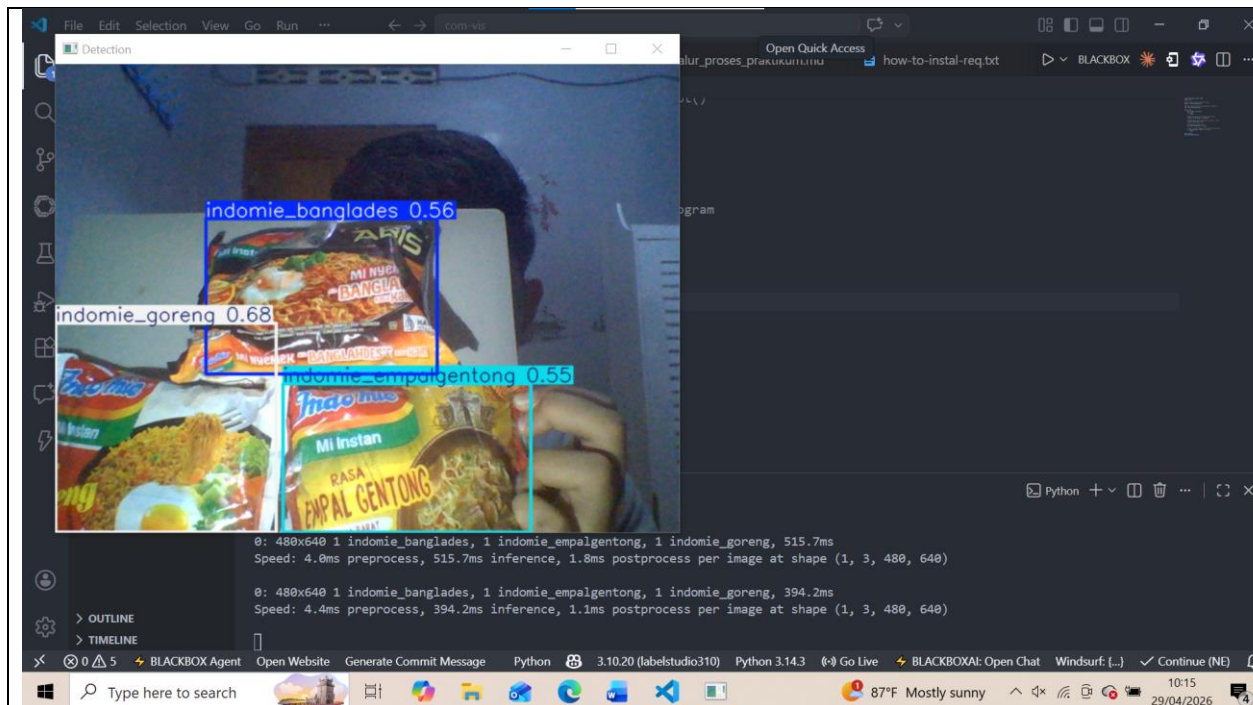
```
cd D:\my_model
```

6. Install library ultralytics

```
pip install ultralytics
```
7. Jalankan file python camera_detect.py

```
python camera_detect.py
```

Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan



Tahap terakhir adalah menjalankan model yang sudah dilatih untuk deteksi langsung lewat kamera laptop.

- Setup: mengekstrak file my_model.zip dan membuat skrip Python camera_detect.py menggunakan *library* ultralytics dan OpenCV (cv2).
- Eksekusi: mengaktifkan kembali *environment* labelstudio310 di Anaconda Prompt, masuk ke direktori folder model, lalu menjalankan perintah python camera_detect.py.
- Hasil Akhir: Seperti yang terlihat di *screenshot*, model berhasil mendeteksi tiga varian Indomie sekaligus (Banglades, Empal Gentong, dan Mie Goreng) secara *real-time*. Meskipun menggunakan kamera laptop dengan pencahayaan seadanya, model tetap bisa memberikan *confidence score* yang stabil di atas 0.50.

D. Tugas Praktikum selanjutnya

1. Cari ide Project Object Detection sederhana dengan menggunakan Yolo kemudian isikan judul ide Project tersebut pada link berikut:
 - a. Kelas INF 6 A : <https://forms.gle/rbr1n6bVCe4iDPoV8>
 - b. Kelas INF 6 B : <https://forms.gle/hCpQzBLe537ZrLV46>
2. Judul dan kelas Dataset harus berbeda dengan teman sekelas
3. Buat Dataset sendiri dengan ketentuan :
 - a. Minimal 200 Gambar / Foto
 - b. Dilarang mengambil dari Repository penyedia dataset seperti Kaggle, Roboflow dll.
 - c. Pembuatan Dataset bisa dilakukan di praktikum selanjutnya, dengan catatan membawa kebutuhan dan peralatannya
4. Realisasikan ide Project Object Detection sederhana tersebut dipertemuan praktikum selanjutnya
5. Yang dikumpulkan berupa dokumentasi Tahapan Project Object Detection dan video presentasi hasil akhir project

proyek deteksi objek untuk tiga jenis kartu: BNI e-money, Kartu Indonesia Sehat (KIS), dan Kartu Tanda Mahasiswa (KTM).

https://drive.google.com/drive/folders/1NyCqg9nQDnDzBylS_eyaINBEGn-DRfa?usp=sharing.

1. Persiapan Data & Analisis Awal

Pada tahap awal, saya fokus mengolah dataset kartu identitas (BNI e-money, KIS, KTM) agar siap dilatih.

- Splitting Dataset: Saya menjalankan skrip `train_val_split.py` untuk membagi 256 gambar secara otomatis menjadi data *train* (230 foto) dan data *validation* (26 foto).
- Distribusi Label: Berdasarkan grafik, jumlah data per kelas sudah cukup seimbang antara BNI e-money (101), KIS (106), dan KTM (77), sehingga model tidak akan berat sebelah saat belajar.
- Kualitas Bounding Box: Dari sebaran data koordinat, terlihat posisi kartu sudah konsisten berada di area tengah gambar dengan ukuran yang pas untuk dideteksi.

2. Proses Pelatihan Model / Training (

Setelah data siap, saya masuk ke tahap inti yaitu melatih otak AI menggunakan arsitektur YOLO.

- Eksekusi Training: Saya melatih model `yolo11s.pt` selama 100 epoch menggunakan GPU Tesla T4 di Google Colab agar prosesnya efisien.
- Evaluasi Grafik: Grafik *results.png* menunjukkan performa yang sangat bagus; nilai *loss* (kesalahan) turun drastis dan tingkat akurasi (*Precision & Recall*) melonjak tajam hingga stabil di atas 90% setelah epoch ke-50.

3. Uji Validasi & Matriks Kebingungan

Di sini saya menguji seberapa pintar model mengenali data baru yang belum pernah dilihat sebelumnya.

- Confusion Matrix: Matriks ini membuktikan model sangat akurat; sebagian besar kartu terdeteksi sesuai kelasnya masing-masing, meskipun ada sedikit *background* yang sempat dianggap objek.
- Hasil Prediksi: Foto-foto validasi menunjukkan model sukses memberikan kotak deteksi (*bounding box*) yang presisi pada berbagai kartu meskipun posisi tangannya berbeda-beda.

4. Implementasi Real-Time

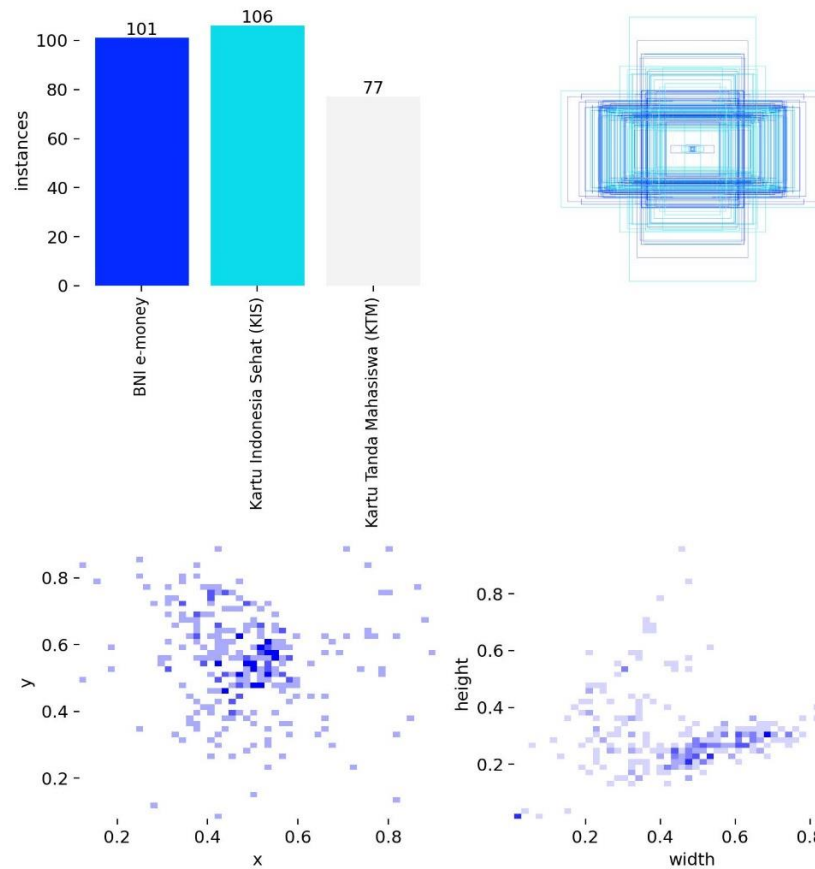
Tahap akhir adalah pembuktian langsung model melalui kamera laptop secara *real-time*.

- Deployment: Saya menjalankan skrip deteksi di perangkat lokal untuk menangkap *input* dari *webcam*.
- Performa Lapangan: Model terbukti sangat responsif dengan kecepatan *inference* sekitar 68ms per frame. Seperti di gambar, model secara bersamaan bisa mengenali BNI e-money (score 0.76) dan KTM (score 0.41) dengan akurasi yang tetap terjaga.

```

Created folder at /content/data/train/images.
Created folder at /content/data/train/labels.
Created folder at /content/data/validation/images.
Created folder at /content/data/validation/labels.
Number of image files: 256
Number of annotation files: 256
Images moving to train: 230
Images moving to validation: 26

```



```

!yolo detect train data=/content/data.yaml model=yolov11s.pt epochs=100 imgsz=640

```

Python

```

Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov11s.pt to 'yolov11s.pt': 100% — 18.4MB 205.4MB/s 0.1s
Ultralytics 8.4.44 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic-rms=False, amp=True, angle=1.0, augment=False, auto-augment-randaugment, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100% — 755.1KB 25.7MB/s 0.0s
Overriding model.yaml nc=80 with nc=3

   from  n  params  module  arguments
0       -1  1      928  ultralytics.nn.modules.conv.Conv  [3, 32, 3, 2]
1       -1  1     18560  ultralytics.nn.modules.conv.Conv  [32, 64, 3, 2]
2       -1  1     26080  ultralytics.nn.modules.block.C3k2  [64, 128, 1, False, 0.25]
3       -1  1     147712  ultralytics.nn.modules.conv.Conv  [128, 128, 3, 2]
4       -1  1     103360  ultralytics.nn.modules.block.C3k2  [128, 256, 1, False, 0.25]
5       -1  1     590336  ultralytics.nn.modules.conv.Conv  [256, 256, 3, 2]
6       -1  1     346112  ultralytics.nn.modules.block.C3k2  [256, 256, 1, True]
7       -1  1    1180672  ultralytics.nn.modules.conv.Conv  [256, 512, 3, 2]
8       -1  1    1380352  ultralytics.nn.modules.block.C3k2  [512, 512, 1, True]
9       -1  1     656896  ultralytics.nn.modules.block.SPPF  [512, 512, 5]
10      -1  1     990976  ultralytics.nn.modules.block.C2PSA  [512, 512, 1]
11      -1  1         0  torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
12     [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat  [1]
13      -1  1     443776  ultralytics.nn.modules.block.C3k2  [768, 256, 1, False]
14      -1  1         0  torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
15     [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat  [1]
16      -1  1     127680  ultralytics.nn.modules.block.C3k2  [512, 128, 1, False]
...
Kartu Tanda Mahasiswa (KTM)      8      8      1      0.808      0.926      0.608
Speed: 0.2ms preprocess, 6.2ms inference, 0.0ms loss, 1.0ms postprocess per image
Results saved to /content/runs/detect/train

```

